EL MODELO ENTIDAD RELACIÓN

SISTEMAS DE BASES DE DATOS 1

Segundo semestre 2026 sección N

EL MODELO ENTIDAD / RELACIÓN

El modelo de datos entidad-relación (E-R) está basado en una percepción del mundo real consistente en objetos básicos llamados entidades y de relaciones entre estas entidades. Hay tres nociones básicas que emplea el modelo de datos E-R: conjuntos de entidades, conjuntos de relaciones y atributos.

DATOS OPERACIONALES

Originalmente se utilizaba el término "datos operacionales" en lugar de "datos persistentes". Y el término "datos operacionales" ya no es del todo apropiado, considerando que en la actualidad una de las propiedades de las bases de datos se refiere a la persistencia en el tiempo de los datos. Debemos tener claro que los datos persistentes pueden ser diarios e históricos.

ENTIDADES

Hemos visto en las unidades anteriores, que en el modelo lógico de la base de datos debemos definir entidades siendo estas objetos de la vida real sobre los cuales queremos almacenar información. Dichas entidades terminan siendo convertidas a tablas cuando pasamos del modelo lógico o Entidad-Relación al modelo físico.

CONJUNTOS DE ENTIDADES

Un conjunto de entidades es un conjunto de entidades del mismo tipo que comparten las mismas propiedades, o atributos. El conjunto de todas las personas que son clientes en un banco, se pueden definir como el conjunto de entidades cliente.

Las entidades individuales que constituyen un conjunto se llaman la extensión del conjunto de entidades. Así, todos los clientes de un banco son la extensión del conjunto de entidades cliente. Una entidad se representa mediante un conjunto de atributos. Los atributos describen propiedades que posee cada miembro de un conjunto de entidades. De igual forma se pueden pensar en la entidad préstamo.

Los posibles atributos del conjunto de entidades cliente son id-cliente, nombre-cliente, calle-cliente y ciudad-cliente. Y los posibles atributos del conjunto de entidades préstamo son número-préstamo e importe. En la vida real, habría más atributos, tales como el número de la calle, el número del portal, la provincia, el código postal, y el municipio, pero no se incluyen en el ejemplo.

Cada entidad tiene un valor para cada uno de sus atributos. Por ejemplo, una entidad cliente en concreto puede tener el valor 32.112.312 para id-cliente, el valor Santos para nombre-cliente, el valor 7ª. calle para calle_cliente y el valor Escuintla para ciudad-cliente. El atributo id-cliente se usa para identificar inequívocamente a los clientes, dado que puede haber más de un cliente con el mismo nombre, calle y ciudad.

Para cada atributo hay un conjunto de valores permitidos, llamados el dominio, o el conjunto de valores, de ese atributo. El dominio del atributo nombre_cliente podría ser el conjunto de todas las cadenas de texto de una cierta longitud. Análogamente, el dominio del atributo número-préstamo podría ser el conjunto de todas las cadenas de la forma «P-n», donde n es un entero positivo.

En la figura se muestra parte de una base de datos de un banco que consta de dos conjuntos de entidades, cliente y préstamo

Santos	32.112.312	Mayor	Peguerinos
Gómez	01.928.374	Carretas	Cerceda
López	67.789.901	Mayor	Peguerinos
Sotoca	55.555.555	Real	Cádiz
Pérez	24.466.880	Carretas	Cerceda
Valdivieso	96.396.396	Goya	Vigo
Fernández	33.557.799	Jazmín	León

cliente

P-17	1.000
P-23	2.000
P-15	1.500
P-14	1.500
P-19	500
P-11	900
P-16	1.300

préstamo

REDUCCIÓN DE UN ESQUEMA E-R A TABLAS

Una base de datos que se basa en un esquema de bases de datos E-R, se puede representar como una colección de tablas. Para cada conjunto de entidades de la base de datos y para cada conjunto de relaciones de la base de datos hay una única tabla a la que se asigna el nombre del conjunto de entidades o del conjunto de relaciones correspondiente.

Cada tabla tiene varias columnas, cada una de las cuales tiene un nombre único. Los modelos E-R y el de bases de datos relacionales son representaciones abstractas y lógicas de empresas del mundo real. Debido a que los dos modelos emplean principios de diseño similares, se puede convertir un modelo E-R en una base de datos relacional.

Convertir una representación de bases de datos de un diagrama E-R a un formato de tablas es la base para la derivación de un diseño de bases de datos relacional desde un diagrama E-R. Aunque existen diferencias importantes entre una relación y una tabla, una relación se puede considerar informalmente como una tabla de valores.

Las restricciones definidas en un diagrama E-R, tales como las llaves primarias y las restricciones de cardinalidad, se corresponden con restricciones sobre las tablas generadas a partir del modelo E-R.

REPRESENTACIÓN DE LOS CONJUNTOS DE ENTIDADES FUERTES

Sea E un conjunto de entidades fuertes con los atributos descriptivos a1, a2,…,aN. Esta entidad se representa mediante una tabla llamada E con n columnas distintas, cada una de las cuales corresponde a uno de los atributos de E. Cada fila de la tabla corresponde a una entidad del conjunto de entidades E.

Considere el conjunto de registros de la tabla préstamo del diagrama E-R mostrado en la figura. Este conjunto de registros tiene dos atributos: número-préstamo e importe. Se representa este conjunto de entidades mediante una tabla llamada préstamo, con dos columnas:

número-préstamo	*importe*
P-11	900
P-14	1.500
P-15	1.500
P-16	1.300
P-17	1.000
P-23	2.000
P-93	500

La fila de la tabla préstamo significa que el número de préstamo P-17 tiene un importe de préstamo de Q1,000. Se puede añadir un nuevo registro a la base de datos insertando una fila en una tabla. También se pueden borrar o modificar las filas. D1 denota el conjunto de todos los números de préstamo y D2 denota el conjunto de todos los saldos.

Cualquier fila de la tabla préstamo debe consistir en una tupla (v1,v2), donde v1 es un número de préstamo (es decir, v1 está definido en el conjunto D1) y v2 es un importe (es decir, v2 está definido en el conjunto D2). En general, la tabla préstamo contendrá sólo un subconjunto del conjunto de todas las posibles filas. El conjunto de todas las filas posibles de préstamo es el producto cartesiano de D1 y D2, denotado por: D1 x D2

REPRESENTACIÓN DE LOS CONJUNTOS DE ENTIDADES DÉBILES

Sea A un conjunto de entidades débiles con los atributos a1, a2,…,am. Sea B el conjunto de entidades fuertes del que A depende. Sea la clave primaria de B el conjunto de atributos b1, b2,…,bn. Se representa el conjunto de entidades A mediante una tabla llamada A con una columna por cada uno de los atributos del conjunto:

número-préstamo	*número-pago*	*fecha-pago*	*importe-pago*
P-11	53	7 junio 2001	125
P-14	69	28 mayo 2001	500
P-15	22	23 mayo 2001	300
P-16	58	18 junio 2001	135
P-17	5	10 mayo 2001	50
P-17	6	7 junio 2001	50
P-17	7	17 junio 2001	100
P-23	11	17 mayo 2001	75
P-93	103	3 junio 2001	900
P-93	104	13 junio 2001	200

La llave primaria del conjunto de la entidad préstamo, de la que depende pago, es número-préstamo. Así, pago se representa mediante una tabla con cuatro columnas etiquetadas con número-préstamo, número-pago, fecha-pago e importe-pago

ATRIBUTOS Y SUS TIPOS

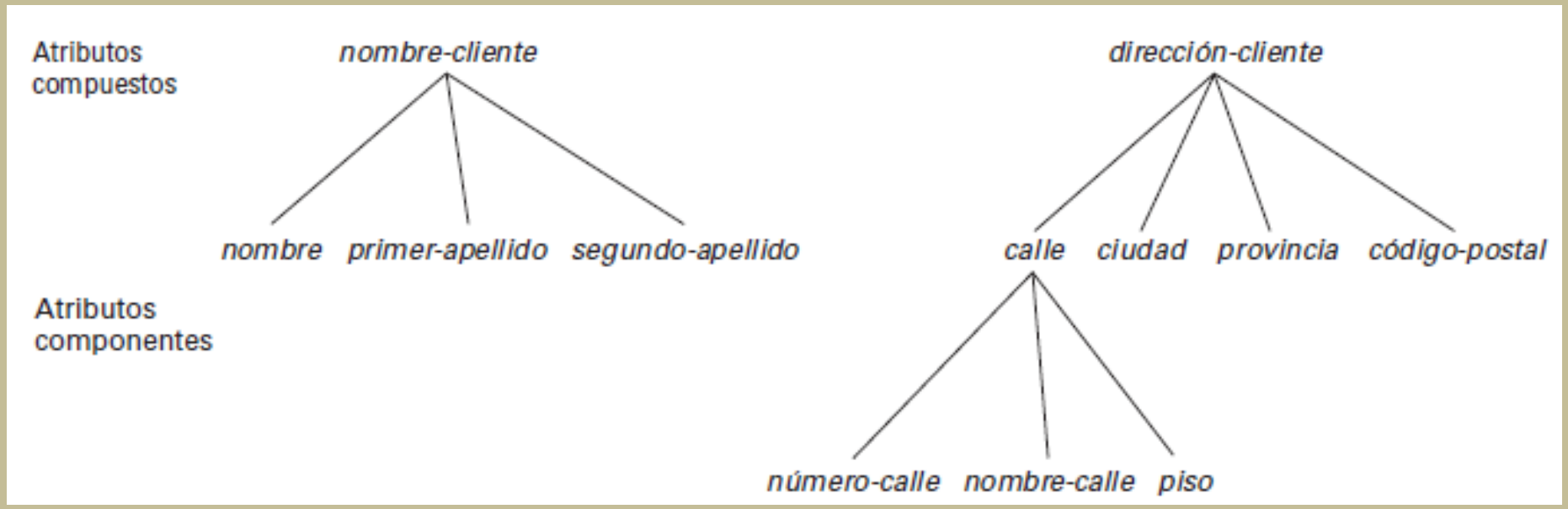
Un atributo, es la propiedad particular de los tipos de entidad como se usa en el modelo E-R y se puede caracterizar por los siguientes tipos de atributo:

- **Atributos simples y compuestos:** En los ejemplos considerados hasta ahora, los atributos han sido simples; es decir, no están divididos en subpartes. Los atributos compuestos, en cambio, se pueden dividir en subpartes (es decir, en otros atributos

Por ejemplo, nombre-cliente podría estar estructurado como un atributo compuesto consistente en nombre, primer-apellido y segundo-apellido. Los atributos compuestos ayudan a agrupar los atributos relacionados, haciendo los modelos más claros.

- **Atributos monovalorados y multivalorados:** Los atributos que se han especificado en los ejemplos tienen todos un sólo valor para una entidad concreta.

Acá podemos ver algunos ejemplos de los atributos compuestos y observar la forma en que estos se pueden disgregar:



- **Atributos derivados:** El valor para este tipo de atributo se puede derivar de los valores de otros atributos o entidades relacionados. Por ejemplo, el conjunto de entidades cliente que tiene un atributo préstamos que representa cuántos préstamos tiene un cliente en el banco. Ese atributo se puede derivar contando el número de entidades préstamo asociadas con ese cliente.

- **Atributo Nulo:** Un atributo puede tomar un valor nulo cuando una entidad no tiene un valor para ese atributo. El valor nulo también puede indicar «no aplicable», es decir, que el valor no existe para la entidad. Por ejemplo, una persona puede no tener segundo nombre de pila y acá el valor nulo puede también designar que el valor de este atributo es desconocido.

LLAVES

Los conceptos de superllave, llave candidata y llave primaria, son aplicables al modelo relacional. Por ejemplo, supongamos un esquema Sucursal, los campos {nombre-sucursal} y {nombre-sucursal, ciudad-sucursal} son superllaves. Ahora {nombre-sucursal, ciudad-sucursal} no puede ser una clave candidata, porque {nombre-sucursal} es un subconjunto de {nombre-sucursal, ciudad-sucursal}

Ahora como vimos anteriormente, {nombre-sucursal} es una superllave. Sin embargo, {nombre-sucursal} es una llave candidata, y servirá también como llave primaria. El atributo ciudad-sucursal no es una superllave, dado que dos sucursales de la misma ciudad pueden tener nombres diferentes (y diferentes de valores).

RELACIONES

El conjunto de relaciones en una base de datos se debe especificar en términos de un lenguaje de definición de datos, de modo que dentro de cualquier relación dada:

- **No existen tuplas duplicadas:** Esta propiedad surge del hecho de que el cuerpo de la relación es un conjunto matemático (de tuplas); los conjuntos en matemáticas no incluyen elementos duplicados.

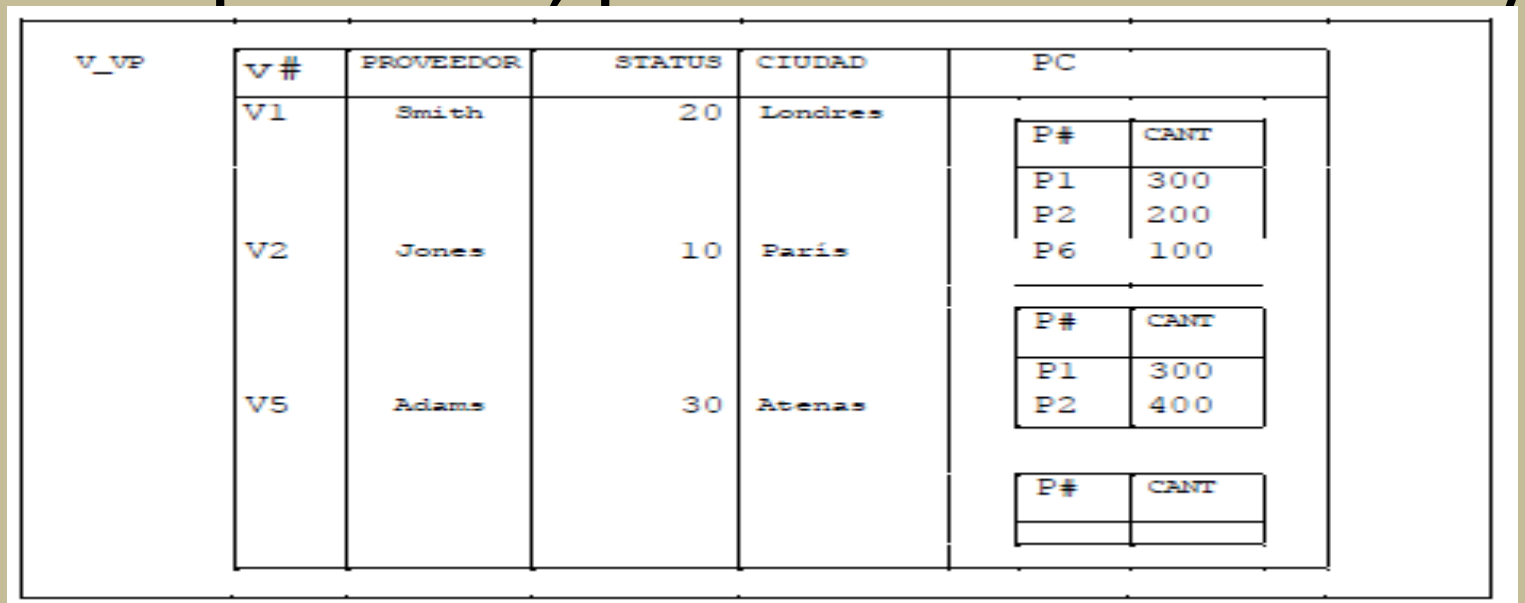
- **Las tuplas están en desorden, de arriba hacia abajo:** Esta propiedad también surge del hecho de que el cuerpo de la relación es un conjunto matemático; en matemáticas, los conjuntos no están ordenados.

- **Los atributos están en desorden, de izquierda a derecha:** Esta propiedad surge del hecho de que el encabezado de una relación también es un conjunto (de atributos).

- **Cada tupla contiene exactamente un valor para cada atributo:** Esta propiedad surge inmediatamente de la definición de una tupla: una tupla es un conjunto de n componentes o pares ordenados de la forma Ai:vi (i= 1,2,..., n). Se dice que una relación que satisface esta cuarta propiedad está normalizada o, lo que es equivalente, está en la primera forma normal.

- **Atributos con valor de relación:** Podemos tener un tipo en particular cuyos valores sean relaciones y por lo tanto tener relaciones con atributos cuyos valores sean a su vez relaciones. En otras palabras, podemos tener relaciones que tengan otras relaciones anidadas dentro de sí mismas.

La figura muestra un ejemplo, la relación V_VP; el atributo PC de esa relación tiene el valor de una relación. Observe en particular que el conjunto vacío de partes que suministra el proveedor V5 está representado por un conjunto vacío (para ser más precisos, por una relación vacía).



V_VP

V#	PROVEEDOR	STATUS	CIUDAD	PC
V1	Smith	20	Londres	P# CANT: P1 300, P2 200
V2	Jones	10	París	P6 100
V5	Adams	30	Atenas	P# CANT: P1 300, P2 400
				P# CANT:

Aquí, el motivo principal para mencionar explícitamente la posibilidad de que los atributos tengan valores de relación es que, históricamente, la mayoría de las publicaciones de bases de datos relacionales han afirmado que dicha posibilidad no es válida (y la mayoría aún lo afirman).

Observe que todos los valores de columnas son atómicos... Es decir, en cada posición de fila y columna de una tabla siempre hay exactamente un valor de datos, nunca un grupo de varios valores. Por lo tanto, tenemos (por ejemplo) en la tabla EMP:

DEPTO#	EMP#
D1 D1	E1 E2

en lugar de

DEPTO#	EMP#
D1	E1 , E2

- **Las relaciones y su interpretación:**

a) El encabezado de cualquier relación dada pueda ser considerado como un predicado.

b) Las tuplas de esa relación puedan ser consideradas como proposiciones verdaderas obtenidas del predicado mediante la sustitución de valores del tipo apropiado por los parámetros o indicadores de posición de ese predicado.

De hecho, la Suposición del Mundo Cerrado (también conocida como la Interpretación del Mundo Cerrado) dice que si una tupla válida —es decir, una que se apegue al encabezado de la relación— no aparece en el cuerpo de la relación, entonces podemos pensar que la proposición correspondiente es falsa. Esto es, el cuerpo de la relación contiene únicamente aquellas tuplas que correspondan a proposiciones verdaderas.

REGLAS DE DISEÑO DEL MODELO ENTIDAD / RELACIÓN

El modelo de entidad/relación es un enfoque ampliamente utilizado para diseñar bases de datos. Es una representación gráfica de entidades, sus atributos y las relaciones entre ellos. Este modelo ayuda a comprender los requisitos de datos de una organización y proporciona una imagen clara de cómo las diferentes entidades se relacionan entre sí.

Hay ciertas reglas de diseño que deben seguirse al crear un modelo de entidad/relación. Estas reglas aseguran que el modelo sea preciso, completo y fácil de entender. Algunas de las reglas de diseño importantes incluyen: cada entidad debe tener un identificador único, cada atributo debe tener un dominio, cada relación debe tener una frase verbal y cada relación debe estar debidamente etiquetada.

Existen algunas reglas básicas que se deben seguir al diseñar un modelo de entidad/relación (ER). A continuación, se mencionan algunas de las más importantes:

1. **Identificar las entidades:** En primer lugar, se deben identificar todas las entidades que estarán presentes en el modelo, así como las relaciones que existen entre ellas.

2. **Definir los atributos:** Para cada entidad, se deben definir los atributos o características que la describen. Estos atributos deben ser relevantes y significativos para la entidad en cuestión.

3. **Definir las relaciones:** Las relaciones entre las entidades deben ser claramente definidas, y se deben especificar las restricciones que se aplican a cada relación.

4. **Definir la cardinalidad y participación:** para determinar la cardinalidad y participación de cada relación en el modelo de base de datos, se debe identificar primero el tipo de relación (1:1, 1:N o N:N) y luego la participación (total o parcial).

5. **Seleccionar las llaves primarias:** Cada entidad debe tener una llave primaria que la identifique de manera única. Las claves primarias deben ser sencillas, estables y relevantes para la entidad.

6. **Normalizar las relaciones:** Se deben aplicar las reglas de normalización para garantizar que las relaciones sean coherentes y libres de redundancias.

7. **Documentar el modelo:** Finalmente, es importante documentar todo el proceso de diseño, incluyendo las decisiones tomadas, las reglas aplicadas y cualquier otra información relevante.

Al seguir estas reglas de diseño, se puede garantizar que el modelo de entidad/relación sea coherente, preciso y fácil de entender.

Esto, a su vez, puede ayudar a mejorar la eficiencia y efectividad de cualquier sistema de bases de datos que se construya a partir de este modelo.

1. Identificar Entidades: El primer paso en el diseño de un modelo entidad-relación consiste en identificar las entidades que formarán parte del sistema.

A continuación, se detallan los pasos y consideraciones que deben seguirse para identificar las entidades de manera efectiva:

a) **Identificar los objetos o conceptos que deben ser modelados en la base de datos:** Se deben analizar los requerimientos del sistema y determinar cuáles son los objetos o conceptos clave que necesitan ser almacenados y gestionados en la base de datos.

b) **Evitar representar redundancias o duplicidades de información:** Al identificar entidades, es fundamental evitar la representación de información redundante o duplicada. La redundancia de datos puede provocar inconsistencias y dificultades en el mantenimiento de la base de datos.

c) **Utilizar sustantivos para nombrar las entidades:** Al asignar nombres a las entidades, es una buena práctica utilizar sustantivos que describan claramente el objeto o concepto que representan. Esto facilita la comprensión del modelo entidad-relación y hace que el diseño sea más legible y fácil de mantener.

Siguiendo estos pasos y consideraciones, se podrá identificar de manera efectiva las entidades en la base de datos y crear un modelo entidad-relación coherente y eficiente. La identificación correcta de entidades es fundamental para garantizar que la base de datos sea fácil de mantener, escalable y proporcione un rendimiento óptimo.

2. Identificar Atributos: Una vez que se han identificado las entidades en el modelo entidad-relación, el siguiente paso es identificar los atributos de cada entidad. Los atributos son las características o propiedades que describen a las entidades y son de vital importancia para la correcta gestión de los datos en la base de datos.

Para identificar los atributos de manera efectiva, es necesario seguir los siguientes pasos y consideraciones:

a) **Identificar las características o propiedades de cada entidad:** Es importante determinar cuáles son las características relevantes que deben almacenarse en la base de datos para cada entidad en el modelo. Estas características pueden ser tanto descriptivas, como nombres o …

direcciones, como cuantitativas, como precios o medidas. Es importante tener en cuenta los requerimientos del sistema y las necesidades de los usuarios al identificar los atributos.

b) **Evitar representar atributos derivados o calculados a partir de otros atributos:** Los atributos derivados son aquellos que pueden ser calculados a partir de otros atributos que ya se encuentran en la base de datos.

En general, se recomienda evitar almacenar atributos derivados en la base de datos para reducir la redundancia y mejorar la eficiencia. En su lugar, estos atributos pueden ser calculados en tiempo real cuando se requieran, utilizando consultas o funciones de la aplicación.

c) **Utilizar sustantivos para nombrar los atributos:** Es recomendable utilizar sustantivos claros y descriptivos al asignar nombres a los atributos de cada entidad. Esto facilita la comprensión del modelo entidad-relación y hace que el diseño sea más legible y fácil de mantener.

La correcta identificación de los atributos de cada entidad es fundamental para garantizar que la base de datos sea fácil de mantener, escalable y proporcione un rendimiento óptimo. Siguiendo estos pasos y consideraciones, se puede identificar de manera efectiva los atributos de cada entidad en el modelo entidad-relación y crear un diseño coherente y eficiente.

3. Identificar Relaciones: Una vez que se han identificado las entidades y sus atributos, el siguiente paso en el diseño de un modelo entidad-relación es identificar las relaciones entre estas entidades. Las relaciones representan las asociaciones entre entidades. A continuación, se detallan los pasos y las consideraciones para identificar relaciones de manera efectiva:

a) **Identificar las asociaciones existentes entre las entidades:** El primer paso para identificar las relaciones es analizar los requerimientos del sistema y las necesidades de los usuarios para determinar cómo se relacionan las entidades entre sí. Las relaciones pueden ser de varios tipos, como relaciones uno a uno, uno a muchos o muchos a muchos.

b) **Utilizar verbos para nombrar las relaciones:** Al asignar nombres a las relaciones, es una buena práctica utilizar verbos que describan claramente la asociación entre las entidades. Por ejemplo, si se está diseñando una base de datos para un sistema de gestión de empleados, una relación entre la entidad "Empleado" y la entidad "Proyecto" podría llamarse "Trabaja_en".

Utilizar verbos claros y descriptivos ayuda a otros diseñadores y usuarios a comprender fácilmente el propósito y la función de cada relación en el sistema.

c) **Asegurarse de que todas las relaciones estén bien definidas y tengan sentido en el contexto del problema:** Es importante revisar cada relación para asegurarse de que esté bien definida y tenga sentido en el contexto del problema. Esto implica verificar que la cardinalidad y la participación de las relaciones sean correctas y que las relaciones se ajusten a las reglas y restricciones del dominio del problema.

Si es necesario, se deben ajustar las relaciones para garantizar que sean coherentes y representen correctamente las asociaciones entre entidades.

Al seguir estos pasos y consideraciones, se puede identificar de manera efectiva las relaciones entre entidades en la base de datos y crear un modelo E-R coherente y eficiente.

La identificación correcta de relaciones es fundamental para garantizar que la base de datos sea fácil de mantener, escalable y proporcione un rendimiento óptimo. Además, la identificación de relaciones adecuadas contribuye a una mejor comprensión del sistema y facilita la implementación de consultas y operaciones en la base de datos.

4. Cardinalidad y Participación: La cardinalidad se refiere a la relación que existe entre dos entidades. Se trata de la cantidad de instancias de una entidad que pueden estar relacionadas con una instancia de la otra entidad. La cardinalidad se puede clasificar en tres tipos principales:

a) **Uno a Uno (1:1):** Indica que una instancia de una entidad está relacionada con exactamente una instancia de la otra entidad, y viceversa. En la notación de líneas y cruces, se representa con una línea recta entre las dos entidades, y un círculo en cada extremo de la línea.

b) **Uno a muchos (1:N):** Indica que una instancia de una entidad puede estar relacionada con varias instancias de la otra entidad, pero cada instancia de la segunda entidad solo puede estar relacionada con una instancia de la primera entidad. En la notación de líneas y cruces, se representa con una línea recta entre las dos entidades, un círculo en un extremo de la línea y una flecha en el otro extremo.

c) **Muchos a muchos (N:N):** Indica que una instancia de una entidad puede estar relacionada con varias instancias de la otra entidad, y viceversa. En la notación de líneas y cruces, se representa con una línea recta entre las dos entidades, y una flecha en cada extremo de la línea.

5. Seleccionar llaves primarias: Seleccionar las claves primarias es una regla básica en el diseño de un modelo de entidad/relación porque las claves primarias son necesarias para identificar de manera única cada entidad en una tabla de la base de datos.

La clave primaria es un conjunto de uno o más atributos que identifican de manera única cada registro en la tabla. La clave primaria se utiliza para garantizar que no se dupliquen los registros y para que cada registro en la tabla pueda ser referenciado desde otras tablas en la base de datos. Al seleccionar una clave primaria, es importante que sea sencilla, estable y relevante para la entidad que representa.

a) **Sencilla:** la clave primaria debe ser fácil de entender y no debe estar compuesta por demasiados atributos para que sea fácil de manejar.

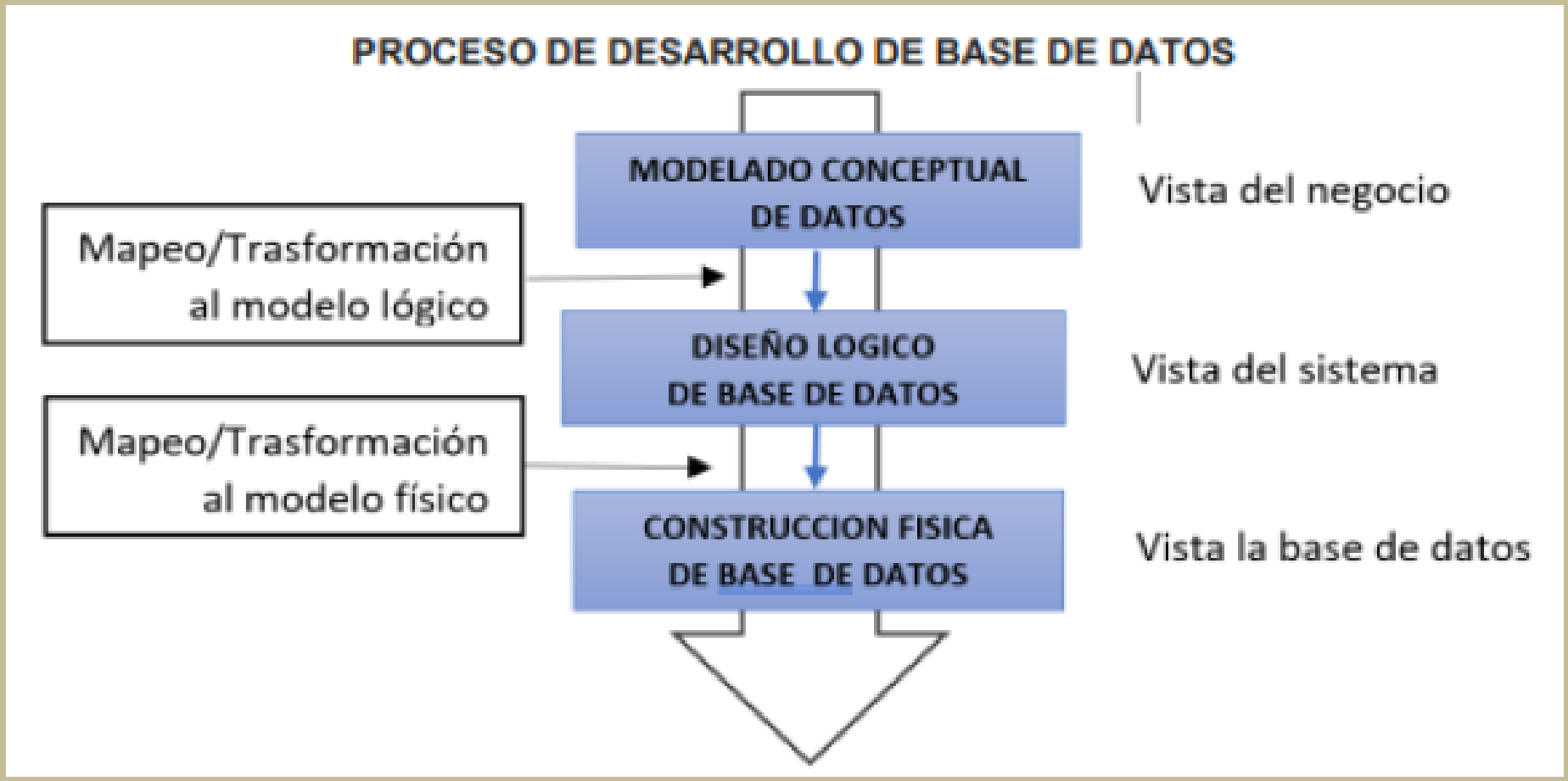
b) **Estable:** la clave primaria debe ser lo más estable posible, lo que significa que no debe cambiar con frecuencia. Esto garantiza que los registros en la tabla sigan siendo únicos y que la integridad de la base de datos se mantenga.

c) **Relevante:** la clave primaria debe ser relevante para la entidad que representa. Debe ser un atributo que tenga sentido en el contexto de la entidad y que tenga un valor único para cada registro.

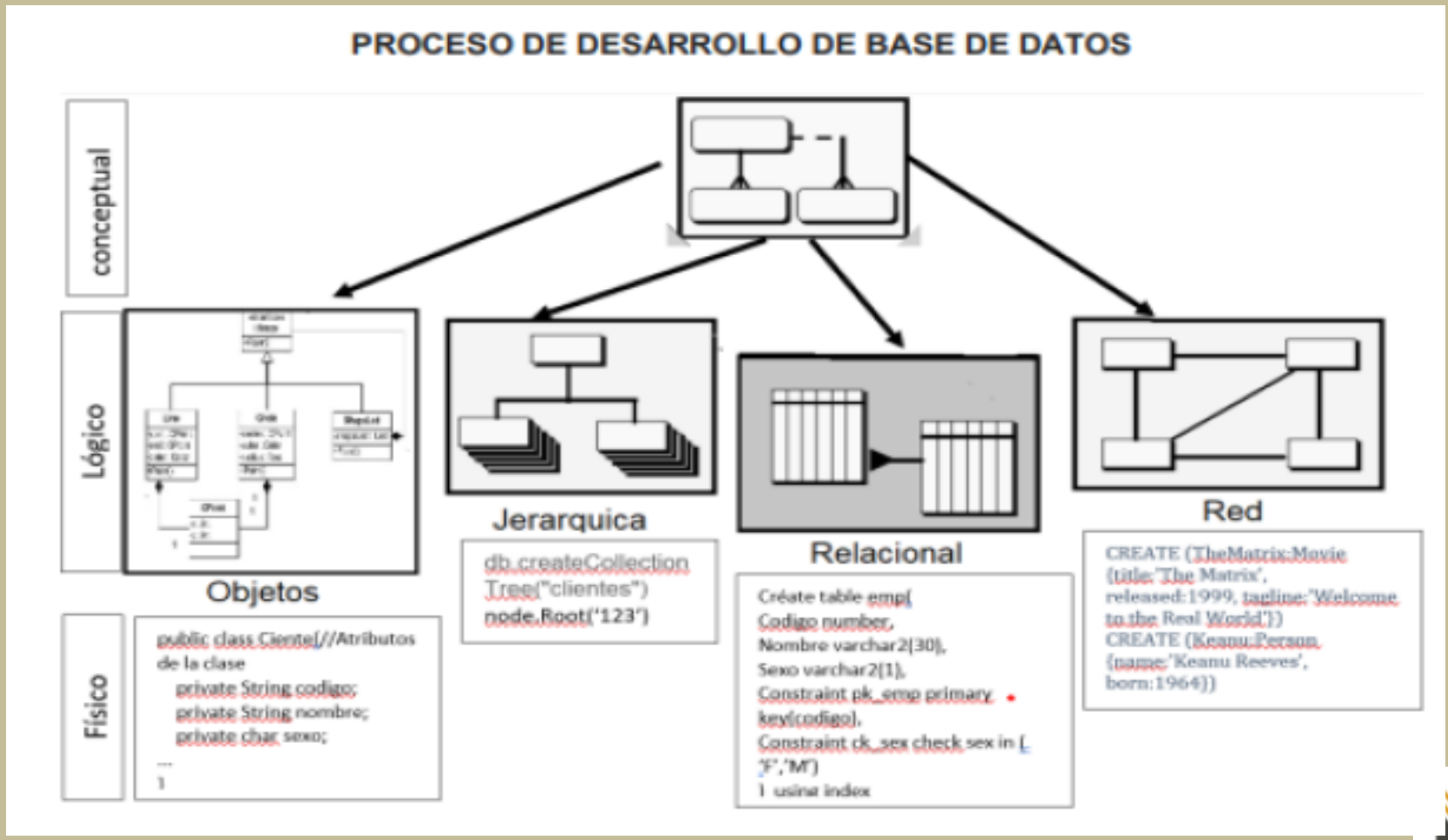
En resumen, seleccionar una clave primaria adecuada es esencial para garantizar la integridad y la consistencia de los datos en la base de datos. La clave primaria debe ser fácil de entender, estable y relevante para la entidad que representa.

EJEMPLOS DE DISEÑO E/R

El último paso en el modelado de datos es transformar el modelo de datos lógico en un modelo de datos físico que organice los datos en una base de datos y que tenga en cuenta los detalles de acceso, rendimiento y almacenamiento. Este es dependiente del software y del hardware.


PROCESO DE DESARROLLO DE BASE DE DATOS
MODELADO CONCEPTUAL DE DATOS
Vista del negocio
Mapeo/Trasformación al modelo lógico
DISEÑO LOGICO DE BASE DE DATOS
Vista del sistema
Mapeo/Trasformación al modelo físico
CONSTRUCCION FISICA DE BASE DE DATOS
Vista la base de datos

En el modelo lógico se especifica el modelo de base de datos a implementar (Relacional, Objetos, Jerárquico, Multidimensional etc.)



Ejemplos de diseño: Repasando el modelo entidad relación consta de los siguientes elementos con su simbología correspondiente.

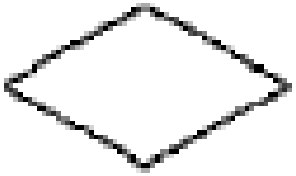
Entidad: Es cualquier tipo de objeto o concepto sobre el que se recoge información: cosa, persona, concepto abstracto o suceso.

Relación: Es una correspondencia o asociación entre dos o más entidades. Cada relación tiene un nombre que describe su función.

Atributo: Es una característica de interés o un hecho sobre una entidad o sobre una relación. Los atributos representan las propiedades básicas de las entidades y las relaciones.



entidad



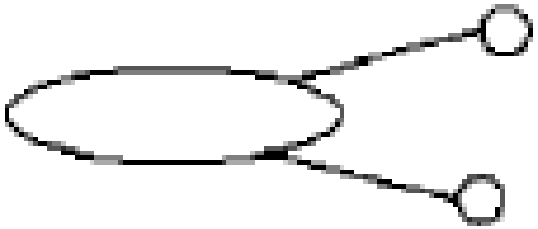
relación



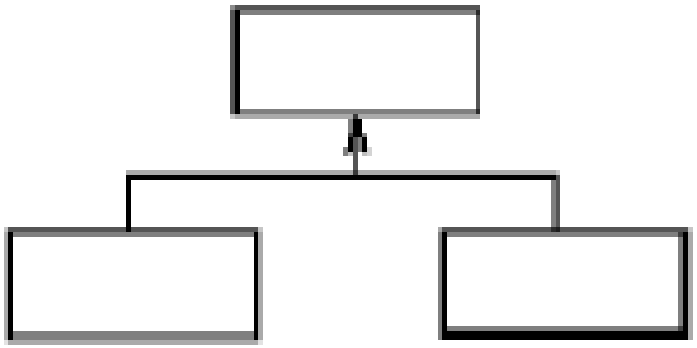
atributo



identificador



atributo compuesto



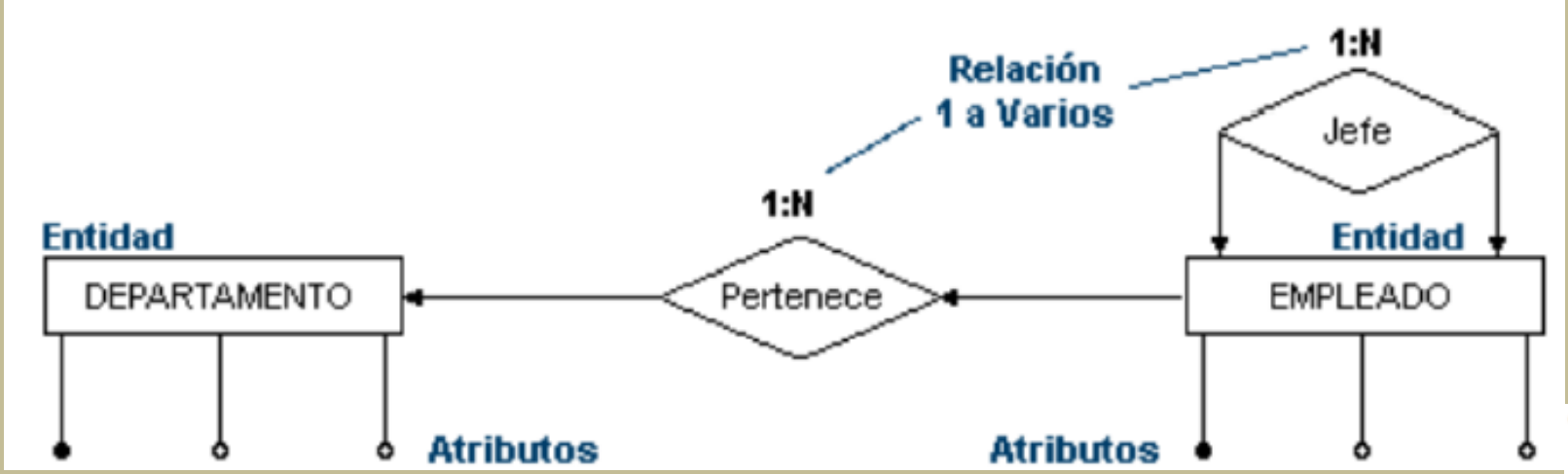
jerarquía de generalización

Identificador: Un identificador de una entidad es un atributo o conjunto de atributos que determina de modo único cada ocurrencia de una entidad.

Jerarquía de generalización: Cuando todas las propiedades de la entidad genérica E son heredadas por las subentidades. Sabiendo esto podemos abstraer la información física de un sistema y representarlo manteniendo la lógica en un modelo entidad relación como se ve a continuación.

Este es un ejemplo sencillo en el que analizamos una metodología de trabajo, la entidad empleado tiene un identificador y varios atributos que se relaciona a un jefe, esta es una relación de uno a muchos ya que un jefe puede tener varios empleados y un empleado solo puede tener un jefe directo, observe que la relación es así misma por que un Jefe puede tener un Jefe y es un empleado a la vez,

Ahora bien sabemos que un empleado solo puede pertenecer a un departamento y un departamento puede tener varios empleados, la entidad de departamento posee varios atributos y un identificador, asi podemos ver como se mantiene la lógica del negocio en este entidad relación.



Técnicas para el modelado E/R:

1. Se debe realizar una descripción textual del problema o sistema de información a automatizar (requerimientos del usuario) ya que debemos entender bien que vamos a modelar.
2. Se hace una lista de los sustantivos y verbos.
3. Los sustantivos son posibles entidades o atributos.

Técnicas para el modelado E/R:

4. Los verbos son posibles relaciones.
5. Analizando las frases se determina la cardinalidad de las relaciones y otros detalles.
6. Se elabora el diagrama entidad-relación.
7. Se completa el modelo con listas de atributos y una descripción de otras restricciones que no se pueden reflejar en el diagrama.

MODELO ENTIDAD RELACIÓN EXTENDIDO

En ocasiones el modelo entidad/relación tradicional tiene ciertas particularidades que son complejas de representar con sus reglas básicas. En base a esto a través del tiempo se han dado agregaciones teniendo como fin aumentar la capacidad expresiva que ya brinda un modelo entidad relación tradicional dando una abstracción necesaria para dar otro tipo de características más específicas

Características:

- Incluye todas las propiedades y características de un Modelo Entidad/Relación tradicional incorporando a sus elementos los conceptos de subclase y superclase con los conceptos de especialización y generalización (de aquí su nombre “Extendido”).

- Incluye un nuevo concepto denominado Categoría.

- Se toma con preponderancia el mecanismo de herencia de atributos.

- Para el Modelo Entidad/Relación Extendido no existe un estándar en su terminología.

Modelos E/R Extendidos notables:

Conforme ha ido evolucionando la literatura en base al modelo entidad relación se han dado una serie de extensiones propias de cada uno de los autores sobre su propia interpretación de un modelo entidad relación extendido.

Modelo ERE de Teorey, Yang y Fry: Establece que la abstracción de las categorías de un modelo Entidad/Relación se dividen en la jerarquía de subconjuntos y de la jerarquía de generalización. Establecen que estas jerarquías de subconjuntos permiten las relaciones entre ellas mientras que las jerarquías de generalización especifican aquellos conjuntos que son estrictamente disjuntos.

Modelo EER de Markowitz y Shoshani: Describe a la generalización como el mecanismo de abstracción por el cual se visualiza a un conglomerado de entidades como un único conjunto de entidades genéricas. También se plantea que la generalización da lugar a una relación transitiva incluyendo que el proceso de generalización es el inverso de la especialización.

Modelo EER de ECER: Este modelo entidad relación conceptual extendido (ECER) se orienta a la implementación de interfaces gráficas para la formulación de consultas y actualizaciones pertinentes. El autor Czejdo establece tres tipos de generalización y/o especialización. El primero envuelve a dos tipos-entidad donde uno es el subconjunto del otro.

El segundo involucra a un tipo-entidad generalizado y a varios tipos-entidad especializados.

El tercero es similar al segundo con la salvedad de que los tipos especializados son disjuntos conectados a un diagrama por la notación d o un más.

Modelo EER de Hohenstein y Gogolla: Los autores pretenden tomar a la generalización y especialización como constructores de tipos con tipos de entrada y salida. Los tipos de entrada se refieren a los ya definidos o básicos que son implementados en los tipos de salida

es decir, que las entidades de los tipos de entrada son distribuidos y agrupados dentro de los tipos de salida. También en su modelo se encuentra tipificado que los atributos puedan tener tipos de datos complejos y enumerados.

DIAGRAMA DE CLASES UML

El diagrama de clases es uno de los diagramas incluidos en UML 2.5 clasificado dentro de los diagramas de estructura y, como tal, se utiliza para representar los elementos que componen un sistema de información desde un punto de vista estático

El diagrama de clases es un diagrama puramente orientado al modelo de programación orientado a objetos, ya que define las clases que se utilizarán cuando se pase a la fase de construcción y la manera en que se relacionan las mismas.

Se podría comparar, salvando las distancias, al famoso diagrama de modelo Entidad-Relación (E/R), pues el UML, tiene una utilidad similar: la representación de datos y su interacción. Ambos diagramas muestran el modelo lógico de los datos de un sistema.

Elementos de un diagrama UML: Los autores pretenden tomar a la generalización y especialización como constructores de tipos con tipos de entrada y salida. Los tipos de entrada se refieren a los ya definidos o básicos que son implementados en los tipos de salida

Clases: Las clases son el elemento principal del diagrama y representa, como su nombre indica, una clase dentro del paradigma de la orientación a objetos. Este tipo de elementos normalmente se utilizan para representar conceptos o entidades del "negocio".

Una clase define un grupo de objetos que comparten características, condiciones y significado. La manera más rápida para encontrar clases sobre un enunciado, sobre una idea de negocio o, en general, sobre un tema concreto es buscar los sustantivos que aparecen en el mismo.

Para representar la clase con estos elementos se utiliza una caja que es dividida en tres zonas utilizando para ello líneas horizontales:

- La primera de las zonas se utiliza para el nombre de la clase. En caso de que la clase sea abstracta se utilizará su nombre en cursiva.

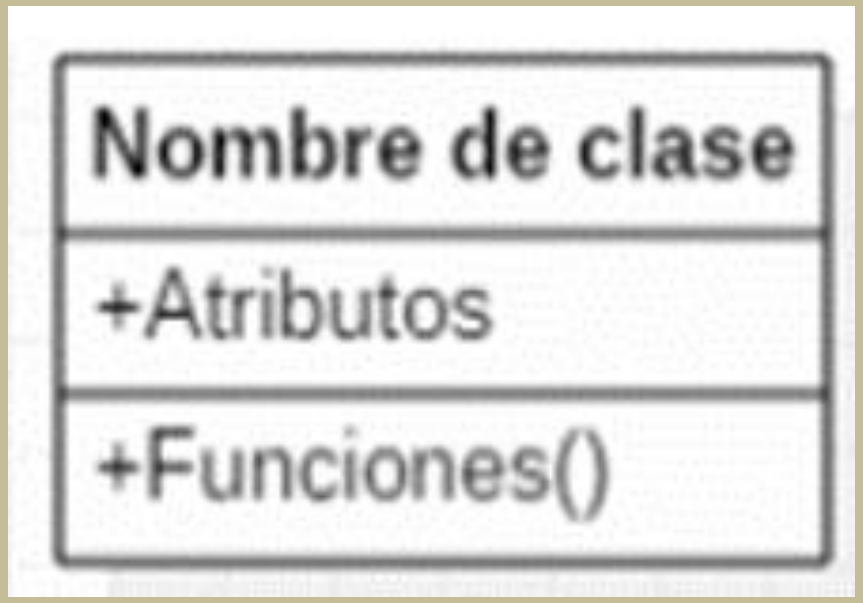
- La segunda de las zonas se utiliza para escribir los atributos de la clase, uno por línea y utilizando el siguiente formato: visibilidad nombre_atributo : tipo = valor-inicial { propiedades }

Aunque esta es la forma “oficial” de escribirlas, es común simplificarlo poniendo el nombre y el tipo o solo el nombre:

- La última de las zonas incluye cada una de las funciones que ofrece la clase. De forma parecida a los atributos, con el siguiente formato: visibilidad nombre_funcion { parametros } : tipo-devuelto { propiedades }

De la misma manera que con los atributos, se suele simplificar indicando únicamente el nombre de la función y, en ocasiones, el tipo devuelto.

Ejemplo de una clase:



Notación de una clase: Tanto los atributos como las funciones incluyen al principio de su descripción la visibilidad. Esta visibilidad se identifica escribiendo un símbolo y podrá ser:

- (+) Pública. Representa que se puede acceder al atributo o función desde cualquier lugar de la aplicación.

- (-) Privada. Representa que se puede acceder al atributo o función únicamente desde la misma clase.

- (#Protegida. Representa que el atributo o función puede ser accedida únicamente desde la misma clase o desde las clases que hereden de ella (clases derivadas).

Estos tres tipos de visibilidad son los más comunes. No obstante, pueden incluirse otros en base al lenguaje de programación que se esté usando (no es muy común). Por ejemplo: (/) Derivado o (~) Paquete.

Relaciones: Una relación identifica una dependencia. Esta dependencia puede ser entre dos o más clases (más común) o una clase hacía sí misma (menos común, pero existen), este último tipo de dependencia se denomina dependencia reflexiva. Las relaciones se representan con una línea que une las clases, esta línea variará dependiendo del tipo de relación.

Las relaciones en el diagrama de clases tienen varias propiedades, que dependiendo la profundidad que se quiera dar al diagrama se representarán o no. Estas propiedades son las siguientes:

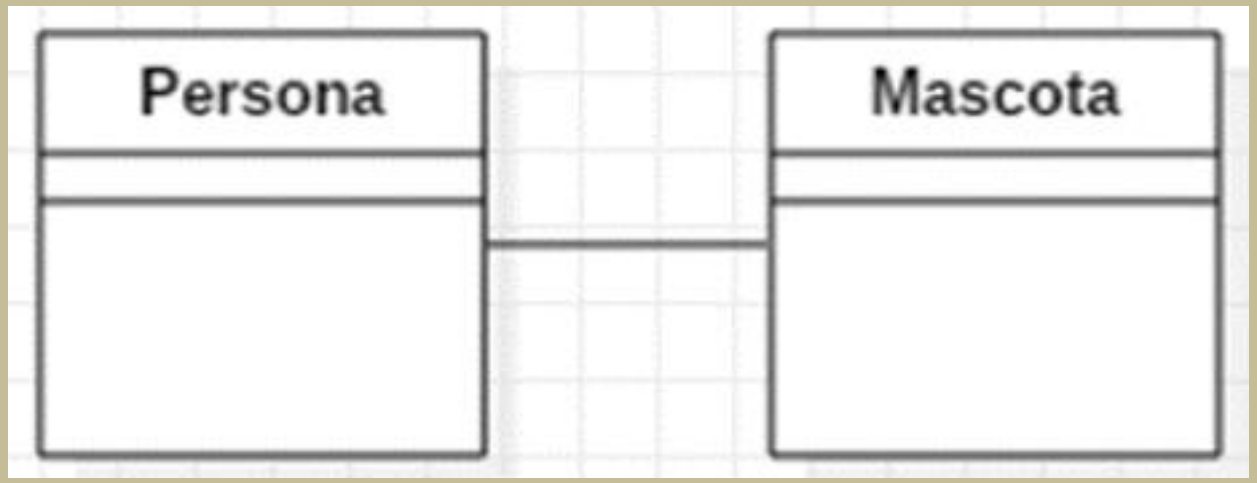
- Multiplicidad: Es decir, el número de elementos de una clase que participan en una relación. Se puede indicar un número, un rango… Se utiliza n o * para identificar un número cualquiera.

- Nombre de la asociación: En ocasiones se escribe una indicación de la asociación que ayuda a entender la relación que tienen dos clases. Suelen utilizarse verbos como por ejemplo: “Una empresa contrata a N empleados”

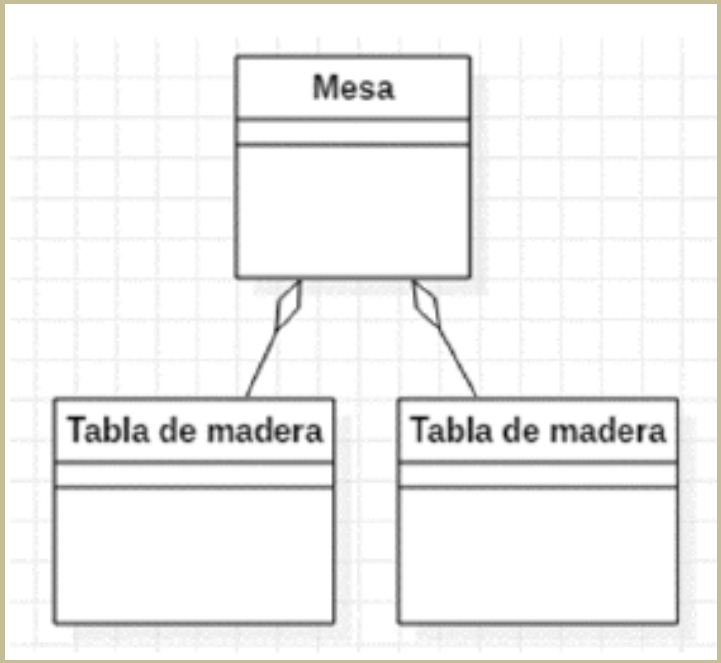
Un diagrama de clases incluye los siguientes tipos de relaciones:

- Asociación.
- Agregación.
- Composición.
- Dependencia.
- Herencia.

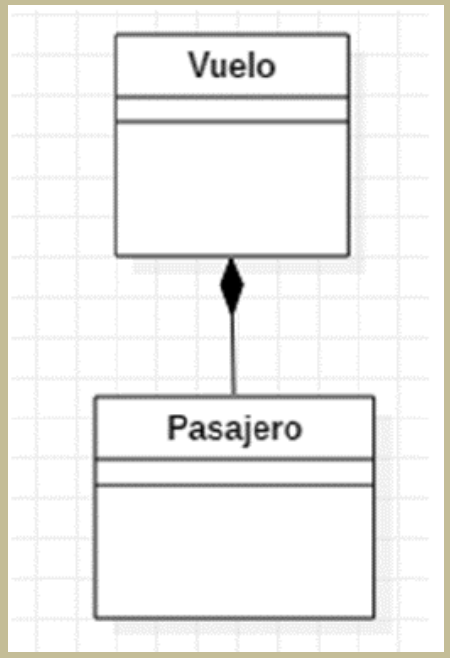
Asociación: Este tipo de relación es el más común y se utiliza para representar dependencia semántica. Se representa con una simple línea continua que une las clases que están incluidas en la asociación. Un ejemplo de asociación podría ser: “Una mascota pertenece a una persona”.



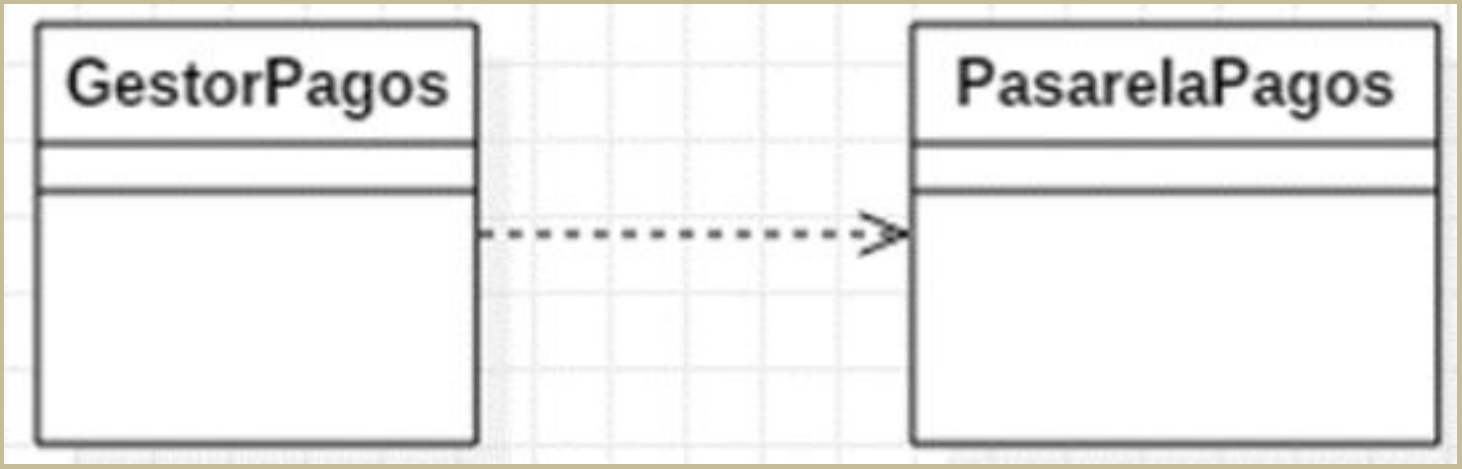
Agregación: Es una representación jerárquica que indica a un objeto y las partes que componen ese objeto. Es decir, representa relaciones en las que un objeto es parte de otro, pero aun así debe tener existencia en sí mismo.



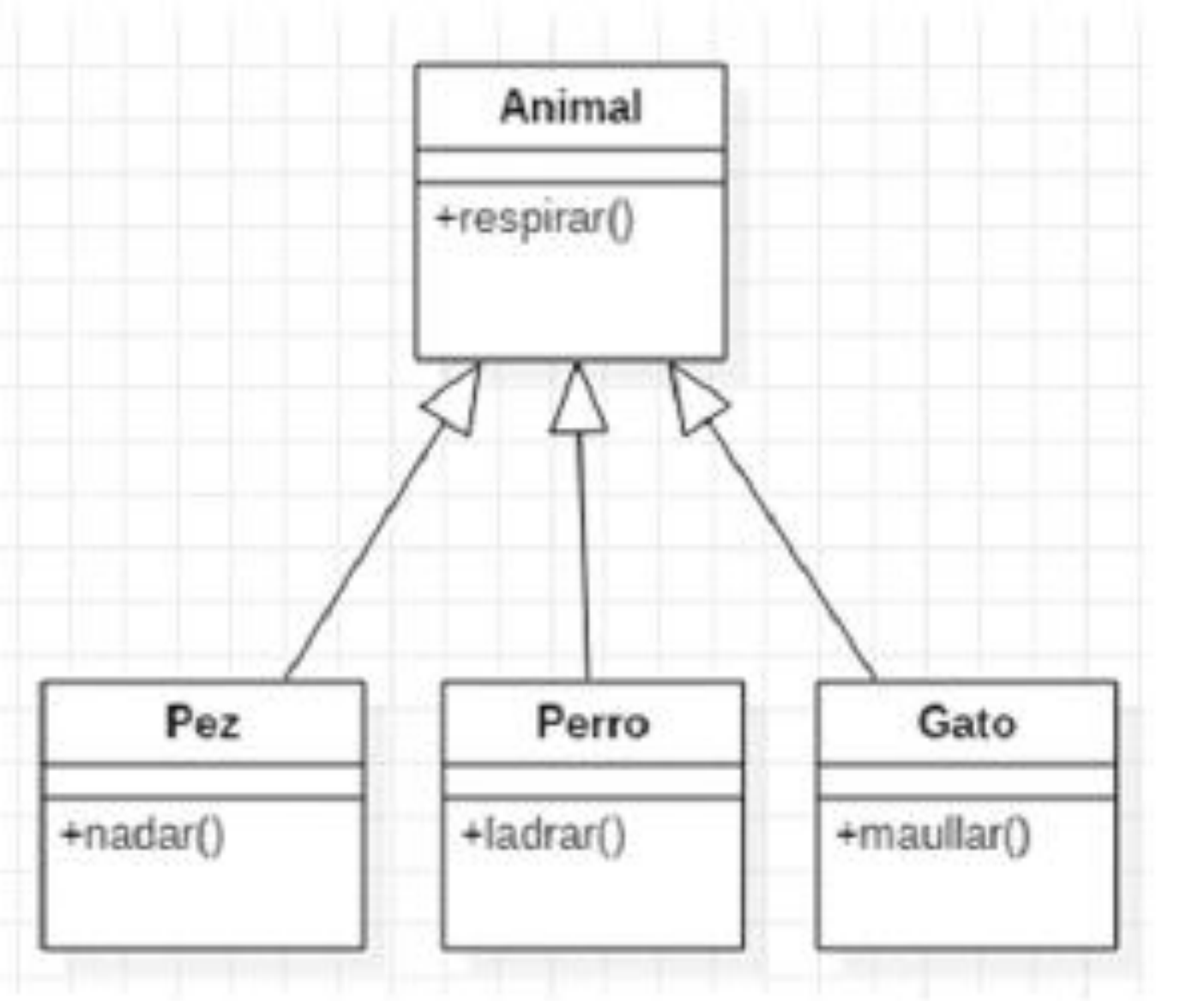
Composición: La composición es similar a la agregación, representa una relación jerárquica entre un objeto y las partes que lo componen, pero de una forma más fuerte. En este caso, los elementos que forman parte no tienen sentido de existencia cuando el primero no existe.



Dependencia: Se utiliza este tipo de relación para representar que una clase requiere de otra para ofrecer sus funcionalidades. Es muy sencilla y se representa con una flecha discontinua que va desde la clase que necesita la utilidad de la otra flecha hasta esta misma.



Herencia: Este tipo de relaciones permiten que una clase (clase hija o subclase) reciba los atributos y métodos de otra clase (clase padre o superclase). Estos atributos y métodos recibidos se suman a los que la clase tiene por sí misma. Se utiliza en relaciones “es un”. Un ejemplo de esta relación podría ser la siguiente: Un pez, un perro y un gato son animales.


Animal
+respirar()
Pez
+nadar()
Perro
+ladrar()
Gato
+maullar()

Interfaces: Una interfaz es una entidad que declara una serie de atributos, funciones y obligaciones. Es una especie de contrato donde toda instancia asociada a una interfaz debe de implementar los servicios que indica aquella interfaz. Dado que únicamente son declaraciones no pueden ser instanciadas. Su representación es similar a las clases, pero indicando arriba la palabra <<interface>>.


«interface»
NombreInterfaz
+Atributos
+Funciones()